

Introduction to Disjoint Set (Union-Find Data Structure)

Definition

- ▶ Two sets are called **disjoint sets** if they **don't have any element in common**.
The disjoint set data structure is used to store such sets. It supports following operations:
- ▶ **Merging** two disjoint sets to a single set using **Union** operation.
- ▶ **Finding** representative of a disjoint set using **Find** operation.
- ▶ These operations are essential for solving problems related to **connectivity**, such as **finding connected components** in a graph or determining whether **two elements are in the same set**.

It is especially useful in problems like:

- ▶ **Connected components** in a graph
- ▶ **Kruskal's algorithm** (for Minimum Spanning Tree)
- ▶ **Network connectivity** (e.g., social networks: "Are A and B in the same group?")
- ▶ **Dynamic connectivity problems**

Core Operations

Find(x):

The task is to find representative of the set of a given element. The representative is always root of the tree. So we implement find() by recursively traversing the parent array until we hit a node that is root (parent of itself).

Union(x, y):

Merges the sets containing x and y.

- Usually done by union by rank/size to keep trees balanced.

Union By Rank

```
def union(self, x, y):  
    root_x = self.find(x)  
    root_y = self.find(y)  
    if root_x != root_y:  
        if self.rank[root_x] < self.rank[root_y]:  
            self.parent[root_x] = root_y  
        elif self.rank[root_x] > self.rank[root_y]:  
            self.parent[root_y] = root_x  
        else:  
            self.parent[root_y] = root_x  
            self.rank[root_x] += 1
```

- ▶ This part uses union by rank (or union by height).
- ▶ It is essentially an estimate of the depth (height) of the tree rooted at a particular element.
- ▶ If `root_x` has a lower rank than `root_y`, it means that the tree rooted at `root_y` is taller, so we want to make the shorter tree (the one with `root_x`) a child of `root_y`. This reduces the potential depth of the tree.

This implementation uses two key optimizations:

- ▶ **Path Compression**
- ▶ **Union by Rank:** In the union method, we attach the tree with a smaller rank to the root of the tree with a larger rank, keeping the tree balanced.

Example: Friend Groups (Connected Components)

Problem:

- ▶ We have **5 people (0-4)** and friendships:
- ▶ $(0, 1)$, $(1, 2)$, $(3, 4)$
- ▶ We want to find how many groups of friends exist.

Step 1: Initialize Disjoint Set

- Each person starts in their **own group**.
- Parent: [0, 1, 2, 3, 4]

Step 2: Process Friendships

- **Union(0,1):**
 - $\text{Root}(0) = 0, \text{Root}(1) = 1 \rightarrow \text{different} \rightarrow \text{merge}.$
- Parent: [0, 0, 2, 3, 4]

Union(1,2):

- $\text{Root}(1) = 0, \text{Root}(2) = 2 \rightarrow \text{different} \rightarrow \text{merge}.$
- Parent: [0, 0, 0, 3, 4]

Union(3,4):

- $\text{Root}(3) = 3, \text{Root}(4) = 4 \rightarrow \text{different} \rightarrow \text{merge}.$
- Parent: [0, 0, 0, 3, 3]

Step 3: Count Groups

- Find unique roots:
- $\text{find}(0)=0, \text{find}(1)=0, \text{find}(2)=0 \rightarrow \text{group 1}$
- $\text{find}(3)=3, \text{find}(4)=3 \rightarrow \text{group 2}$
- ☒ **Number of groups = 2**

Scenario: Social Network (Friend Circles / Communities)

- ▶ In a social network:
- ▶ **People = Nodes**
- ▶ **Friendship = Edge**
- ▶ We want to find:
 - ▶ How many friend groups exist?
 - ▶ Are two people in the same group?
 - ▶ Merge groups when new friendships are added.
- ▶ **Union-Find** is perfect here because it efficiently answers "**Are X and Y connected?**" and "**Connect X and Y**".

Example:

- ▶ 5 users $\rightarrow$ 0, 1, 2, 3, 4
Friendships:

- ▶ (0,1), (1,2), (3,4)

Step 1: Initially (no friendships)

- ▶ Each person is in their own group:
- ▶ Groups = {0}, {1}, {2}, {3}, {4}

Step 2: Process Friendships

- ▶ $\text{union}(0,1) \rightarrow \text{merge} \rightarrow \{0,1\}, \{2\}, \{3\}, \{4\}$
- ▶ $\text{union}(1,2) \rightarrow \text{merge} \rightarrow \{0,1,2\}, \{3\}, \{4\}$
- ▶ $\text{union}(3,4) \rightarrow \text{merge} \rightarrow \{0,1,2\}, \{3,4\}$

Step 3:

Result

- ✓ Groups = 2 ($\{0,1,2\}$ and $\{3,4\}$)
- ✓ Query: $\text{find}(0) == \text{find}(2) \rightarrow \text{True}$ (same circle)
- ✓ Query: $\text{find}(0) == \text{find}(4) \rightarrow \text{False}$ (different circles)

Code

```
def find(self, x):  
    if self.parent[x] != x:  
        self.parent[x] = self.find(self.parent[x]) # path compression  
    return self.parent[x]
```

Memory management techniques and garbage collection logic in data structures

Memory Management Techniques in Data Structures:

Memory management deals with how memory is **allocated, used, and freed** when working with data structures.

(a) Static Allocation

- Memory is fixed at **compile time**.
- Example: Arrays in C.
- Advantage: **Fast access**.
- Disadvantage: **Wastage** if unused or insufficient if too small.

(b) Dynamic Allocation

- Memory is allocated at **runtime** (using ``malloc``, ``new``, etc.).
- Example: Linked Lists, Trees, Graphs.
- Advantage: Flexible, grows/shrinks as needed.
- Disadvantage: Slower allocation, possible **fragmentation**.

(c) Stack Allocation

- Used for **function calls and local variables**.
- Memory automatically freed when function exits.
- Efficient but limited in size.

(d) Heap Allocation

- Used for **dynamic data structures** (linked list, dynamic arrays, etc.).
- Manual management required (freeing memory) unless garbage collection is present.

(e) Memory Pooling (Slab Allocation)

- Preallocates fixed-size blocks for frequently used data structures.
- Faster and reduces fragmentation.
- Used in OS kernels and databases.

2. Garbage Collection Logic

- ▶ Garbage collection (GC) is the process of automatically reclaiming memory that is no longer in use.
- ▶ This is common in languages like Java, Python, C#, etc

Types of Garbage Collection:

1. Reference Counting

- ▶ Each object has a counter of references.
- ▶ When count = 0, object is freed.
- ▶ Problem: Can't handle circular references (object $A \rightarrow B$, $B \rightarrow A$).

2. Mark and Sweep

- ▶ GC starts from root objects, marks all reachable objects, then sweeps (deletes) unmarked ones.
- ▶ Handles circular references.
- ▶ Disadvantage: Stop-the-world pauses.

3. Copying Collector

- ▶ Divides memory into two halves.
- ▶ Live objects are copied to the new half, old half is cleared.
- ▶ Advantage: Eliminates fragmentation.
- ▶ Disadvantage: Requires extra space.

4. Generational Garbage Collection

- ▶ Objects are divided into "young" and "old" generations.
- ▶ Most objects die young, so frequent collection happens in young space.
- ▶ Old objects are collected less often.
- ▶ Used in JVM, .NET CLR.

Memory management: Page replacement algorithms

- ▶ Page Replacement Algorithm is used when a **page fault** occurs. Page Fault means the page referenced by the **CPU is not present** in the main memory.

Page Replacement Algorithms

1. FIFO (First-In-First-Out) Page Replacement

- ▶ Idea: The **oldest loaded page** (first inserted) is removed when cache is full.
- ▶ Suitable Data Structure: Queue
- ▶ Enqueue new pages.
- ▶ Dequeue the oldest when capacity is full.

Example: Consider the Pages referenced by the CPU in the order are 6, 7, 8, 9, 6, 7, 1, 6, 7, 8, 9, 1

Pages >>	6	7	8	9	6	7	1	6	7	8	9	1
Frame 3			8	8	8	7	7	7	7	7	9	9
Frame 2		7	7	7	6	6	6	6	6	8	8	8
Frame 1	6	6	6	9	9	9	1	1	1	1	1	1
	Miss	Miss	Miss	Miss	Miss	Miss	Miss	Hit	Hit	Miss	Miss	Hit

Number of Page Faults = 9

Optimal Page Replacement

- ▶ In this algorithm, the page which would be used after the longest interval is replaced.
- ▶ Consider the Pages referenced by the CPU in the order are 6, 7, 8, 9, 6, 7, 1, 6, 7, 8, 9, 1, 7, 9, 6

Pages >>	6	7	8	9	6	7	1	6	7	8	9	1	7	9	6
Frame 3			8	9	9	9	1	1	1	1	1	1	1	1	1
Frame 2		7	7	7	7	7	7	7	7	7	7	7	7	7	7
Frame 1	6	6	6	6	6	6	6	6	6	8	9	9	9	9	6
	Miss	Miss	Miss	Miss	Hit	Hit	Miss	Hit	Hit	Miss	Miss	Hit	Hit	Hit	Miss

The number of Page Faults = 8

Least Recently Used

- ▶ This algorithm works on previous data. The page which is used the earliest is replaced
- ▶ Example: Consider the Pages referenced by the CPU in the order are 6, 7, 8, 9, 6, 7, 1, 6, 7, 8, 9, 1, 7, 9, 6

Pages>>	6	7	8	9	6	7	1	6	7	8	9	1	7	9	6
Frame 3			8	8	8	7	7	7	7	7	7	1	1	1	6
Frame 2		7	7	7	6	6	6	6	6	6	9	9	9	9	9
Frame 1	6	6	6	9	9	9	1	1	1	8	8	8	7	7	7
	Miss	Miss	Miss	Miss	Miss	Miss	Miss	Hit	Hit	Miss	Miss	Miss	Miss	Hit	Miss

The number of Page Faults = 12

Memory Management

- Memory management is a critical function that ensures **efficient allocation and utilization of memory resources**. When processes are initiated, they require execution time and storage space, which is allocated in memory blocks.
- Memory allocation algorithms are used to manage the assignment of memory to these incoming processes.

The following are the most used algorithms:

1. First Fit
2. Best Fit
3. Worst Fit

First-Fit Memory Allocation

- ▶ The **First Fit** method works by searching through memory blocks from the beginning, looking for the first block that can fit the process.
- ▶ First Fit simply finds the first available block that can fit the process and assigns it there. It's a straightforward and fast way to allocate memory.

First Fit Allocation in OS

Process Sizes	90	50	30	40
	P1	P2	P3	P4

First FIT Allocation

20	100	40	200	10
Block 1	Block 2	Block 3	Block 4	Block 5

	Size	Allocated	Memory Wastage After Process Occupies
Process 1	90	Block 2	$100 - 90 = 10$
Process 2	50	Block 4	$200 - 50 = 150$
Process 3	30	Block 3	$40 - 30 = 10$
Process 4	40	Unallocated	-

P4 remains
Unallocated

Disadvantages

- ▶ First Fit can lead to both external fragmentation
- ▶ For large processes, First Fit can be less efficient

Worst-Fit Memory Allocation

- ▶ In **Worst-Fit Memory Allocation** allocation technique, the process traverses the whole memory and always search for the **largest hole/partition**, and then the process is placed in that hole/partition. It is a slow process because it has to traverse the entire memory to search the largest hole.

Example

- ▶ **Available Memory Blocks:** 100K, 500K, 200K, 300K, 600K
- ▶ **Processes to Allocate:** 212K, 417K, 112K

Solution

- ▶ **Process 1 (212K):** The algorithm selects the largest available block, 600K, for the 212K process.
- ▶ **Process 2 (417K):** The largest remaining block is 500K, so it's allocated here.
- ▶ **Process 3 (112K):** The largest remaining block that fits is 388K (which was the original 600K minus the 212K process).

Drawbacks

1. Wastage of large blocks:
2. Poor utilization
3. Search overhead

Best Fit

- ▶ In Best Fit, the process is placed in the smallest available block that is large enough to hold it.
- ▶ The idea is to reduce leftover space (internal fragmentation).

Best-Fit Example

Input : blockSize[] = {100, 500, 200, 300, 600};

processSize[] = {212, 417, 112, 426};

Output:

Process No.	Process Size	Block no.	Remaining
1	212	4	88
2	417	2	83
3	112	3	88
4	426	5	176

Drawbacks

- ▶ External fragmentation
- ▶ High search overhead